

Αλγόριθμοι και Πολυπλοκότητα

ΦΡΟΝΤΙΣΤΗΡΙΑΚΕΣ ΑΣΚΗΣΕΙΣ (4)

Διαίρει και βασίλευε (2/2)

9/4/2024

Έφη Μαλέσιου
Διδακτορική φοιτήτρια



ΑΣΚΗΣΗ 6

- Ε0, 2022-23
- (εναλλακτική προσέγγιση)

Υπολογίστε την πολυπλοκότητα χρόνου των ακόλουθων αλγορίθμων:

Algorithm 1

```
1: begin algorithm
2:    $arg \leftarrow -1$ 
3:   for  $i \leftarrow 1$  to  $2n$  with step 1 do
4:     for  $j \leftarrow i$  to  $i^2$  with step 1 do
5:        $arg \leftarrow CALC(j)$ 
6:     end for
7:   end for
8: end algorithm
9:
10: procedure  $CALC(w)$ 
11:    $res \leftarrow 0$ 
12:   for  $i \leftarrow 1$  to  $w^{0.5}$  with step 0.1 do
13:      $res \leftarrow res + \log(i)$ 
14:   end for
15:   return  $res$ 
16: end procedure
```

ΑΣΚΗΣΗ 6

- Ε0, 2022-23

Λύση για τον Αλγόριθμο 1

- 1^ο for : $2n$ βήματα $= O(n)$
- 2^ο for : $i^2 - i + 1$ βήματα, με $\max_i = 2n \Rightarrow 4n^2 - 2n + 1$ βήματα, άρα $= O(n^2)$
- υπολογισμός $\arg - \text{CALC}$: $10 \cdot w^{0.5}$ βήματα, με $\max_w = 4n^2 \Rightarrow 10 \cdot 2n$ βήματα, άρα $= O(n)$, αφού ο υπολογισμός του res είναι $O(1)$

Έτσι, συνολικά έχει πολυπλοκότητα: $O(n) \cdot O(n^2) \cdot O(n) = O(n^4)$, γιατί οι διαδικασίες είναι εμφωλευμένες.

$$\sum_{i=1}^{2n} \sum_{j=i}^{i^2} 10 \cdot \sum_{c=1}^{\sqrt{j}} 1 = 10 \cdot \sum_{i=1}^{2n} \sum_{j=i}^{i^2} \sqrt{j}$$

$$(i^2 - i + 1) \cdot \sqrt{i} \leq \sum_{j=i}^{i^2} \sqrt{j} = \sqrt{i} + \sqrt{i+1} + \dots + \sqrt{i^2-1} + \sqrt{i^2} \leq (i^2 - i + 1) \cdot i$$

$\Theta(i^{2.5})$ $\Theta(i^3)$

$$\sum_{i=1}^{2n} i^3 - i^2 + i = n^2(2n+1)^2 - \frac{n(2n+1)(4n+1)}{3} + n(2n+1) = \Theta(n^4)$$

$$\sum_{i=1}^{2n} \sum_{j=i}^{i^2} 10 \cdot \sum_{c=1}^{\sqrt{j}} 1 = 10 \cdot \sum_{i=1}^{2n} \sum_{j=i}^{i^2} \sqrt{j} = O(n^4)$$

Θέμα 4 [15 μονάδες]

Να βρείτε την πολυπλοκότητα των παρακάτω αλγορίθμων. Να δοθεί σύντομη αιτιολόγηση.

Algorithm 1

```
1: begin algorithm
2:    $arg \leftarrow 1$ 
3:   for  $i \leftarrow 1$  to  $n$  with step 1 do
4:     for  $j \leftarrow 1$  to  $i$  with step 1 do
5:        $arg \leftarrow CALC(j)$ 
6:     end for
7:   end for
8: end algorithm
9:
10: procedure  $CALC(m)$ 
11:    $i \leftarrow 1$ 
12:    $s \leftarrow 1$ 
13:   while  $s \leq m$  do
14:      $i \leftarrow i + 1$ 
15:      $s \leftarrow s + i$ 
16:   end while
17:   return  $s$ 
18: end procedure
```

- $m_{\max}(\cdot)$
- πολυπλοκότητα πράξεων μεταξύ αριθμών = $O(1)$
- καλύτερο άνω φράγμα while, σχέση τιμών i, s / σχέση i με τον αριθμό επαναλήψεων

		w1	w2	w3	w4
i	1	2	3	4	5
s	1	3	6	10	15

- πιο καλό O , η πορεία αιτιολόγησης το υποδεικνύει

Algorithm 2

```
1: begin algorithm
2:    $arg \leftarrow 0$ 
3:   for  $i \leftarrow 1$  to  $n$  with step 1 do
4:     for  $j \leftarrow 1$  to  $n$  with step  $(2 \cdot j)$  do
5:        $arg \leftarrow CALC(j)$ 
6:     end for
7:   end for
8: end algorithm
9:
10: procedure  $CALC(m)$ 
11:    $s \leftarrow m$ 
12:   while  $s \leq (2 \cdot m)$  do
13:      $s \leftarrow s + 1$ 
14:   end while
15:   return  $s$ 
16: end procedure
```

- $j = 1$
- while $j \leq n$
 - $arg \leftarrow \mathbf{CALC(j)}$
 - $j = j + 2j = \mathbf{3j}$
- $m_{\max}(i)$
- για διευκόλυνση, μπορούμε να υποθέσουμε ότι $n = 3^k$

$$T(n) = \sqrt{n} T(\sqrt{n}) + n$$

$$\Theta(;))$$

$$S(n) = \frac{T(n)}{n} = \frac{\sqrt{n} T(\sqrt{n}) + n}{n} = \frac{T(\sqrt{n})}{\sqrt{n}} + 1$$

$$S(n) = S(\sqrt{n}) + 1$$

Φ2 - σελ. 24: $T'(n) = T'(\sqrt{n}) + 1 = \Theta(\log \log n)$

Θ. κυριαρχίας - περίπτωση 2:

$$S(n) = \frac{T(n)}{n} = \Theta(\log \log n) \Rightarrow T(n) = \Theta(n \log \log n)$$

$$T(n) = \begin{cases} 1, n = 1 \\ 2 \cdot T\left(\frac{n}{2}\right) + n, n > 1 \end{cases}$$

Μέθοδος Αντικατάστασης (ακριβής λύση)

Υπόθεση: $T(n) = n \lg n + n = n(\lg n + 1)$

Επαγωγή:

Βάση: $n = 1 \Rightarrow n \lg n + n = 1 = T(n)$

Υπόθεση: $T(k) = k \lg k + k$
για κάθε $k < n$

* $\lg = \log_2$

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n \\ &= 2\left(\frac{n}{2} \lg \frac{n}{2} + \frac{n}{2}\right) + n && \text{(επαγωγική υπόθεση, } n/2 < n) \\ &= n \lg \frac{n}{2} + n + n \\ &= n(\lg n - \lg 2) + n + n \\ &= n \lg n - n + n + n \\ &= n \lg n + n. \end{aligned}$$

$$T(n) = 8 T\left(\frac{n}{2}\right) + \Theta(n^2)$$

Ο, με Αντικατάσταση

Άνω όριο: $T(n) \leq 8 T\left(\frac{n}{2}\right) + cn^2, c > 0$

Υπόθεση 1: $T(n) \leq dn^3, d > 0$

$$\begin{aligned} T(n) &\leq 8d(n/2)^3 + cn^2 \\ &= 8d(n^3/8) + cn^2 \\ &= dn^3 + cn^2 \\ &\not\leq dn^3 \text{ δεν «δουλεύει»} \end{aligned}$$

Υπόθεση 2: $T(n) \leq dn^3 - d'n^2, d, d' > 0$

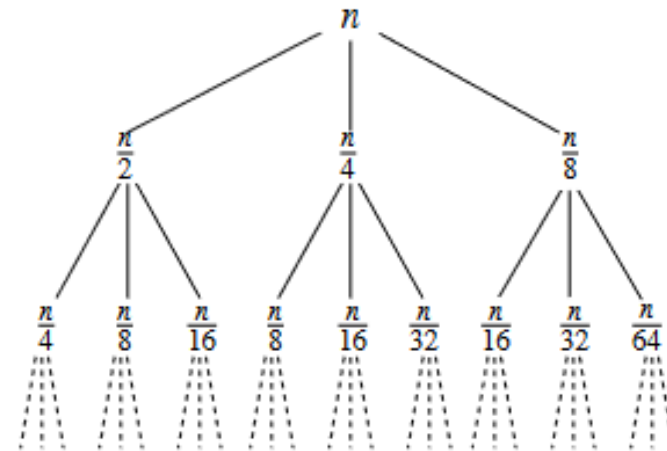
$$\begin{aligned} T(n) &\leq 8(d(n/2)^3 - d'(n/2)^2) + cn^2 \\ &= 8d(n^3/8) - 8d'(n^2/4) + cn^2 \\ &= dn^3 - 2d'n^2 + cn^2 \\ &= dn^3 - d'n^2 - d'n^2 + cn^2 \\ &\leq dn^3 - d'n^2 \quad \text{αν} \quad \begin{matrix} -d'n^2 + cn^2 \leq 0, \\ d' \geq c \end{matrix} \end{aligned}$$

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + T\left(\frac{n}{8}\right) + n$$

Άνω όριο

Υπόθεση: $T(n) \leq cn, c > 0$

$$\begin{aligned} T(n) &= T(n/2) + T(n/4) + T(n/8) + n \\ &\leq cn/2 + cn/4 + cn/8 + n \\ &= 7cn/8 + n \\ &= (1 + 7c/8)n \\ &\leq cn \quad \text{αν} \quad c \geq 8. \end{aligned}$$



$$\left. \begin{array}{c} n \\ n/2 \\ n/4 \\ n/8 \\ \vdots \end{array} \right\} \log_8 n$$

$$\begin{aligned} n \left(\frac{4+2+1}{8} \right) &= \frac{7}{8}n \\ n \left(\frac{1}{4} + \frac{2}{8} + \frac{3}{16} + \frac{2}{32} + \frac{1}{64} \right) &= n \frac{16+16+12+4+1}{64} \\ &= n \frac{49}{64} = \left(\frac{7}{8} \right)^2 n \\ &\vdots \end{aligned}$$

Υποθέστε ότι είστε σύμβουλοι σε μία τράπεζα που την ενδιαφέρει ο εντοπισμός οικονομικών εγκλημάτων, και έρχονται σε σας με το ακόλουθο πρόβλημα. Έχουν μία συλλογή από n τραπεζικές κάρτες που έχουν κατάσχει επειδή υποπτεύονται ότι χρησιμοποιούνται σε απάτες. Κάθε **τραπεζική κάρτα** είναι ένα μικρό πλαστικό αντικείμενο που περιέχει μία μαγνητική ταινία με κάποια κρυπτογραφημένα δεδομένα και αντιστοιχεί σε ένα μοναδικό τραπεζικό λογαριασμό. Κάθε **λογαριασμός** μπορεί να έχει πολλές τραπεζικές κάρτες που αντιστοιχούν σε αυτόν, και θα λέμε ότι δύο τραπεζικές κάρτες είναι **ισοδύναμες** αν αντιστοιχούν στον ίδιο λογαριασμό.

Είναι πολύ δύσκολο να διαβαστεί ο τραπεζικός λογαριασμός άμεσα από την κάρτα, όμως η τράπεζα διαθέτει μία υψηλής τεχνολογίας ‘συσκευή ελέγχου ισοδυναμίας’ που δέχεται δύο κάρτες και μετά από την εκτέλεση κάποιων ($O(1)$) υπολογισμών, προσδιορίζει αν είναι ισοδύναμες.

Το **ερώτημά** τους είναι το ακόλουθο:

Σε ένα σύνολο S από n κάρτες $c_1, c_2, \dots, c_n$, υπάρχει ένα σύνολο με περισσότερες από $\frac{n}{2}$ κάρτες ($> \frac{n}{2}$) που είναι ισοδύναμες μεταξύ τους και άρα ύποπτες;

Υποθέστε ότι οι **μόνες δυνατές λειτουργίες** που μπορείτε να εκτελέσετε με τις κάρτες είναι η επιλογή δύο καρτών και η τοποθέτησή τους στη συσκευή **ελέγχου ισοδυναμίας** η οποία επιστρέφει TRUE αν είναι ισοδύναμες αλλιώς FALSE.

Ένας **απλοϊκός αλγόριθμος** που έχουν ήδη σκεφτεί στην τράπεζα είναι ο εξής: Θα ελέγχει με τη σειρά τις κάρτες για το αν κάποια από αυτές είναι ύποπτη. Αυτό επιτυγχάνεται αν ελέγχουμε κάθε κάρτα σε σχέση με όλες τις $n - 1$ υπόλοιπες. Αν βρεθεί μία ύποπτη κάρτα τότε αναφέρουμε την κάρτα και σταματάμε. Αν ελέγχθηκαν $\frac{n}{2}$ κάρτες και καμία δεν ήταν ύποπτη τότε δεν υπάρχει τέτοιο σύνολο ισοδύναμων καρτών που να έχει πληθάρημο $> \frac{n}{2}$ και ο αλγόριθμος σταματά και επιστρέφει NIL. Ο αλγόριθμος έχει πολυπλοκότητα $O(n^2)$, αφού στη χειρότερη περίπτωση θα ελεγχθούν $\frac{n}{2}$ κάρτες όπου κάθε μία θα ελεγχθεί με τις υπόλοιπες $n - 1$. Ο αλγόριθμος αυτός δεν είναι αποδεκτός από την τράπεζα λόγω της υψηλής πολυπλοκότητάς του.

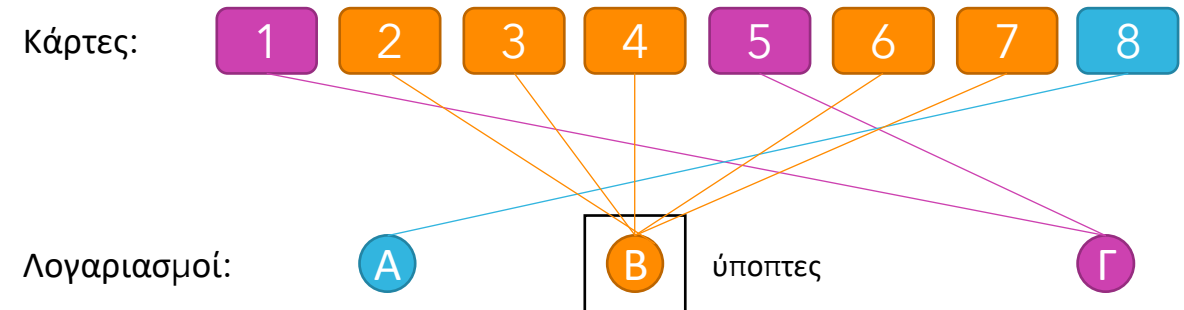
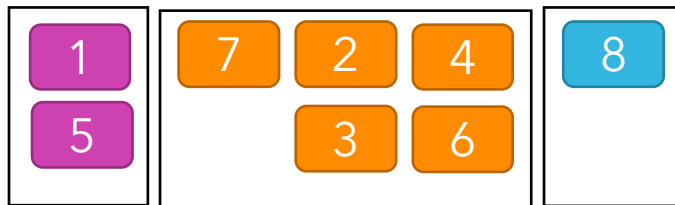
Σας ζητείται να σχεδιάσετε και να διατυπώσετε σε **φυσική γλώσσα** έναν **πιο αποδοτικό** αλγόριθμο χρησιμοποιώντας τη μέθοδο **διαίρει και βασίλευε**. Ο αλγόριθμος αρκεί να επιστρέφει μία από τις ύποπτες κάρτες αν υπάρχει τέτοιο σύνολο ή NIL αν δεν υπάρχει τέτοιο σύνολο καρτών.

Υπόδειξη: Χωρίστε τις κάρτες σε κλάσεις ισοδυναμίας, δηλαδή σε κάρτες που αντιστοιχούν στον ίδιο λογαριασμό, και ελέγξτε αν σε κάθε υποπρόβλημα υπάρχει μία κλάση που είναι ύποπτη, συνδυάζοντας κατάλληλα τις λύσεις.

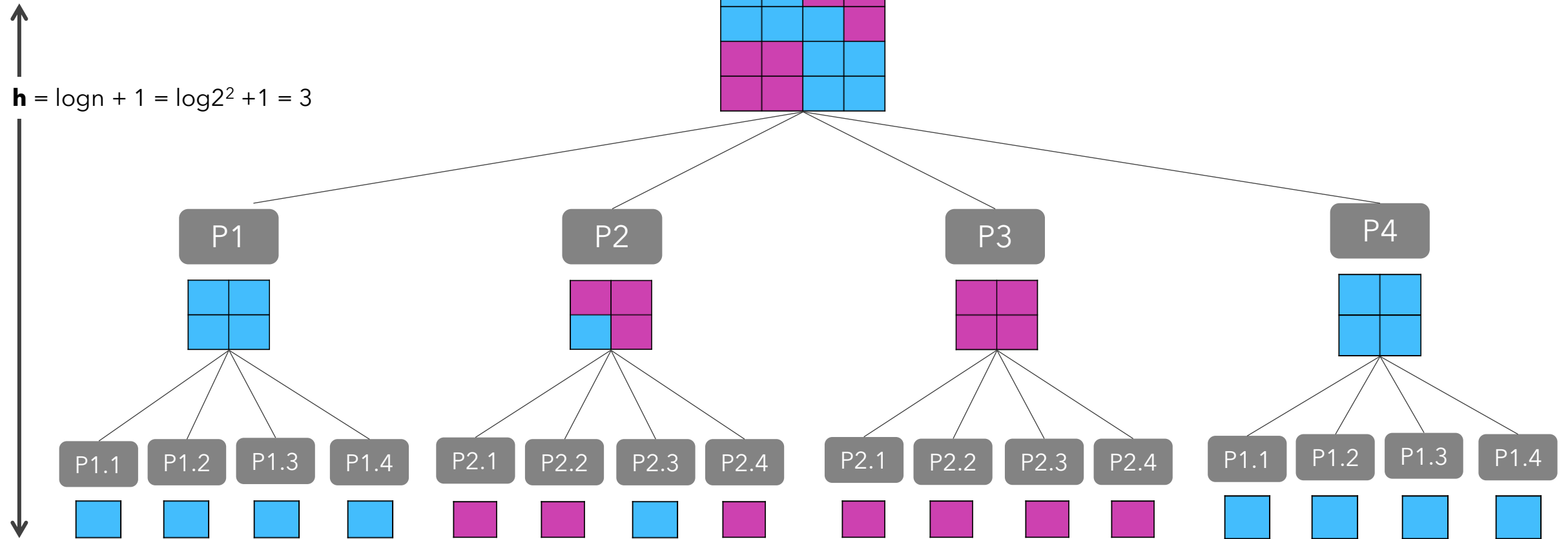
Ισοδυναμία( , ) = TRUE

Ισοδυναμία( , ) = FALSE

Κλάσεις
ισοδυναμίας:



Μεταφορά του προβλήματος εύρεσης του κυρίαρχου χρώματος σε 1 διάσταση



Χωρίζουμε τις κάρτες του συνόλου S σε κλάσεις ισοδυναμίας καρτών $e_1, e_2, \dots, e_k$, όπου $1 \leq k \leq n$, και κάθε κλάση αντιστοιχεί σε έναν λογαριασμό τραπέζης οπότε περιέχει όλες τις ισοδύναμες κάρτες. Ο αλγόριθμος χωρίζει το σύνολο S με τις κάρτες σε δύο σύνολα $c_1, \dots, c_{\frac{n}{2}}$ και $c_{\frac{n}{2}+1}, \dots, c_n$. Έπειτα, εκτελείται αναδρομικά και στα δύο σύνολα και αν σε κάποιο από αυτά τα σύνολα έχει βρεθεί μία κλάση ισοδυναμίας με πληθάνημο μεγαλύτερο από το μισό του μεγέθους του συνόλου τότε επιστρέφεται μία κάρτα από αυτή την κλάση. Αν υπάρχει μία κλάση ισοδυναμίας e_i με περισσότερες από $\frac{n}{2}$ κάρτες τότε σίγουρα σε ένα από τα δύο σύνολα θα υπάρχουν περισσότερες από τις μισές κάρτες που θα ανήκουν στην e_i . Επομένως, τουλάχιστον μία από τις δύο αναδρομικές κλήσεις θα επιστρέψουν μία κάρτα που να ανήκει στην e_i . Το αντίστροφο αυτής της πρότασης δεν ισχύει. Μπορεί να επιστραφεί μία κάρτα από ένα από τα δύο υποσύνολα αλλά αυτή να μην αντιστοιχεί σε κλάση ισοδυναμίας που να πλειοψηφεί στο αρχικό σύνολο μιας και πλειοψηφούσε μόνο στο ένα από τα δύο. Επομένως, όταν επιστρέφεται μία κάρτα από μία αναδρομική κλήση θα πρέπει να ελέγχεται αν είναι όντως πλειοψηφούσα κάρτα στην τρέχουσα αναδρομική κλήση. Ακολουθεί η περιγραφή του αλγορίθμου σε φυσική γλώσσα με βήματα:

- Αν $|S| = 1$ τότε επιστρέφεται η μοναδική κάρτα
- Αν $|S| = 2$ τότε
 - έλεγχος αν οι δύο κάρτες είναι ισοδύναμες
 - αν είναι, επιστρέψε μία από τις δύο
- Έστω S_1 το σύνολο των πρώτων $\lfloor \frac{n}{2} \rfloor$ καρτών
- Έστω S_2 το σύνολο των υπολοίπων καρτών
- Αναδρομική κλήση του αλγορίθμου για το σύνολο S_1

- Αν επιστραφεί μία κάρτα c
 - έλεγε τη c σε σχέση με όλες τις κάρτες στο S
- Αν δεν έχει βρεθεί ακόμα πλειοψηφούσα κάρτα στο S τότε
 - Αναδρομική κλήση του αλγορίθμου για το σύνολο S_2
 - Αν επιστραφεί μία κάρτα c'
 - * έλεγε τη c' σε σχέση με όλες τις κάρτες στο S
- Επιστρέψε μία πλειοψηφούσα κάρτα αν βρέθηκε αλλιώς NIL

Η ορθότητα του αλγορίθμου προκύπτει από την παρατήρηση ότι αν υπάρχει πλειοψηφούσα κλάση ισοδυναμίας τότε αυτή θα ανακαλυφθεί ότι είναι πλειοψηφούσα σε τουλάχιστον μία από τις δύο αναδρομικές κλήσεις. Έστω ότι το πλήθος των ελέγχων για n κάρτες είναι $P(n)$. Ο αλγόριθμος εκτελεί το πολύ δύο αναδρομικές κλήσεις και για κάθε μία εκτελεί το πολύ n ελέγχους. Άρα, και για τις δύο αναδρομικές κλήσεις εκτελούνται το πολύ $2n$ έλεγχοι. Άρα η παρακάτω αναδρομική σχέση δίνει ένα πάνω φράγμα στην απόδοση του αλγορίθμου: $P(n) = 2P(\frac{n}{2}) + 2n$. Εφαρμόζοντας το Θεώρημα Κυριαρχίας παίρνουμε ότι $P(n) = O(n \log n)$.

Ψευδοκώδικας:

Θεωρείται πως όλες οι κάρτες είναι αποθηκευμένες μέσω κάποιου αναγνωριστικού σε έναν πίνακα T , n θέσεων. Η συνάρτηση CHECK, εκτελεί έλεγχο ισοδυναμίας μεταξύ 2 καρτών.

```
1: procedure DC_CHECK(T, n)
2:   check  $\leftarrow$  FALSE
3:   card  $\leftarrow$  NIL
4:   if  $n = 1$  then
5:     return  $T[1]$ 
6:   end if
7:   if  $n = 2$  then
8:     check  $\leftarrow$  CHECK( $T[1]$ ,  $T[2]$ )
9:     if check then
10:      return  $T[1]$ 
11:    else
12:      return NIL
13:    end if
14:   end if
15:   card  $\leftarrow$  DC_CHECK( $T[1, \dots, \frac{n}{2}]$ ,  $\frac{n}{2}$ )
16:   if card  $\neq$  NIL then
17:     score  $\leftarrow$  1
18:      $i \leftarrow 1$ 
19:     while  $i \leq n$  do
20:       if num  $\neq i$  then
21:         check  $\leftarrow$  CHECK( $T[num]$ ,  $T[i]$ )
22:       end if
23:       if check then
24:         score  $\leftarrow$  score + 1
25:       end if
26:        $i \leftarrow i + 1$ 
27:     end while
```

```
28:   if score  $\leq \frac{n}{2}$  then
29:     card  $\leftarrow$  NIL
30:   end if
31: else
32:   card  $\leftarrow$  DC_CHECK( $T[\frac{n}{2} + 1, \dots, n]$ ,  $\frac{n}{2}$ )
33:   if card  $\neq$  NIL then
34:     score  $\leftarrow$  1
35:      $i \leftarrow 1$ 
36:     while  $i \leq n$  do
37:       if num  $\neq i$  then
38:         check  $\leftarrow$  CHECK(card,  $T[i]$ )
39:       end if
40:       if check then
41:         score  $\leftarrow$  score + 1
42:       end if
43:        $i \leftarrow i + 1$ 
44:     end while
45:   end if
46:   if score  $\leq \frac{n}{2}$  then
47:     card  $\leftarrow$  NIL
48:   end if
49: end if
50: return card
51: end procedure
```

- αναδρομικός
- διαίρει και βασίλευε
- «**pivot**» - στοιχείο περιστροφής
 - στη **σωστή** τελική θέση
 - **αριστερά**: μικρότερα
 - **δεξιά**: μεγαλύτερα

Επιλογή «**pivot**» - στοιχείου περιστροφής

Πρώτο στοιχείο

2	6	5	3	8	7	1	0
---	---	---	---	---	---	---	---

- Χειρότερη: $O(n^2)$
 - $T(n) = T(n-1) + cn$
- Μέση: $O(n \log n)$

Διάμεσος-3

2	6	5	3	8	7	1	0	4
---	---	---	---	---	---	---	---	---

2	6	5	3	4	7	1	0	8
---	---	---	---	---	---	---	---	---

2	6	5	3	4	7	1	0	8
---	---	---	---	---	---	---	---	---

Τυχαίο στοιχείο

2	6	5	3	8	7	1	0
---	---	---	---	---	---	---	---

Randomized quicksort

- Αναμένεται, με μεγάλη πιθανότητα: $O(n \log n)$

2	6	5	3	8	7	1	0
---	---	---	---	---	---	---	---

2	6	5	3	8	7	1	0
---	---	---	---	---	---	---	---

pivot

2	6	5	0	8	7	1	3
---	---	---	---	---	---	---	---

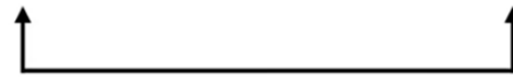
- αναζήτηση από **Αριστερά, Μεγαλύτερου**
- αναζήτηση από **Δεξιά, Μικρότερου**

2	6	5	0	8	7	1	3
---	---	---	---	---	---	---	---

↑
itemFromLeft

↑
itemFromRight

2	1	5	0	8	7	6	3
---	---	---	---	---	---	---	---



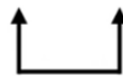
itemFromRight



2	1	5	0	8	7	6	3
---	---	---	---	---	---	---	---

↑
itemFromLeft

2	1	0	5	8	7	6	3
---	---	---	---	---	---	---	---



index right < index left => **Stop**

itemFromRight



2	1	0	5	8	7	6	3
---	---	---	---	---	---	---	---

↑
itemFromLeft

itemFromRight



2	1	0	3	8	7	6	5
---	---	---	---	---	---	---	---

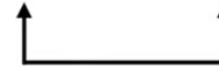
↑
itemFromLeft

Quicksort - Γρήγορη ταξινόμηση

Συνέχεια έως την ταξινόμηση όλων

2	1	0	3	8	7	6	5
---	---	---	---	---	---	---	---

8	7	6	5
---	---	---	---



2	1	0	3	6	5	7	8
---	---	---	---	---	---	---	---

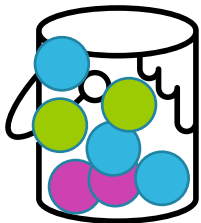
0	1	2	3	5	6	7	8
---	---	---	---	---	---	---	---

Δίνονται ένα ρομπότ & ένα καλάθι με χρωματιστές σφαίρες.

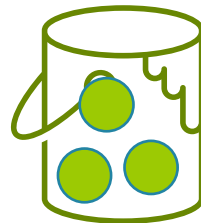
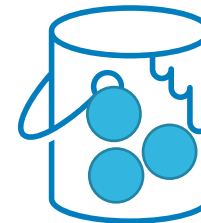
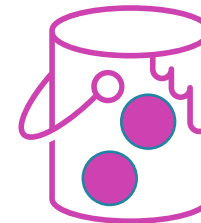
Οι σφαίρες έχουν ένα από τα χρώματα: κόκκινο (0), μπλε (1), πράσινο (2)

Θέλουμε το ρομπότ να ταξινομήσει χρωματικά τις σφαίρες τοποθετώντας τις σε 3 άδεια καλάθια: στο πρώτο τις κόκκινες (0), στο δεύτερο τις μπλε (1) και στο τρίτο τις πράσινες (2).

- Σχεδιάστε γραμμικό επιτόπιο αλγόριθμο για το πρόβλημα
- Εξηγήστε πώς μια λύση του προβλήματος αυτού μπορεί να χρησιμοποιηθεί στο αλγόριθμο quicksort (γρήγορης ταξινόμησης).



0	1	2	2	1	0	0	2	1	1
									



- Αρχική κατάσταση:

0	1	2	2	1	0	0	2	1	1
---	---	---	---	---	---	---	---	---	---

- Κατάσταση – στόχος:

0	0	0	1	1	1	1	2	2	2
---	---	---	---	---	---	---	---	---	---

- Χρησιμοποιούνται 3 δείκτες:
 - pivot (αρχικά 1) – διατρέχει τον πίνακα, ένα στοιχείο τη φορά, ξεκινώντας από την πρώτη θέση
 - low (αρχικά 1) – μεταφορά στοιχείων στην κατάλληλη θέση (swap)
 - high (αρχικά N = μήκος πίνακα) – μεταφορά στοιχείων στην κατάλληλη θέση (swap)
- Σταδιακά, στον πίνακα έχουμε αριστερά τα στοιχεία με τιμή 0, μετά αυτά με τιμή 1, κάποια «άγνωστα» στα οποία ακόμα δεν έχει φτάσει το pivot και τα στοιχεία με τιμή 2 δεξιά.
- Στην εκάστοτε θέση του pivot:
 - Αν το στοιχείο είναι 0, γίνεται ανταλλαγή των στοιχείων της θέσης του pivot και του low, και οι δείκτες low και pivot αυξάνονται κατά 1.
 - Αν το στοιχείο είναι 1, αυξάνεται ο δείκτης pivot κατά 1. Δε γίνεται ανταλλαγή.
 - Αν το στοιχείο είναι 2, γίνεται ανταλλαγή των στοιχείων της θέσης του pivot και του high, ο δείκτης pivot αυξάνεται κατά 1 και ο δείκτης high μειώνεται κατά 1.

p -> pivot
l -> low
h -> high

Παράδειγμα:



l

h

p



l

h

p



l

h

p



l

h

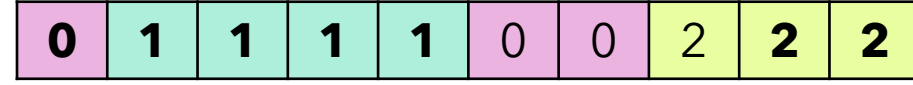
p



l

h

p



l

h

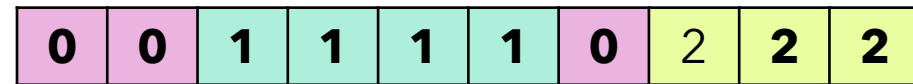
p



l

h

p

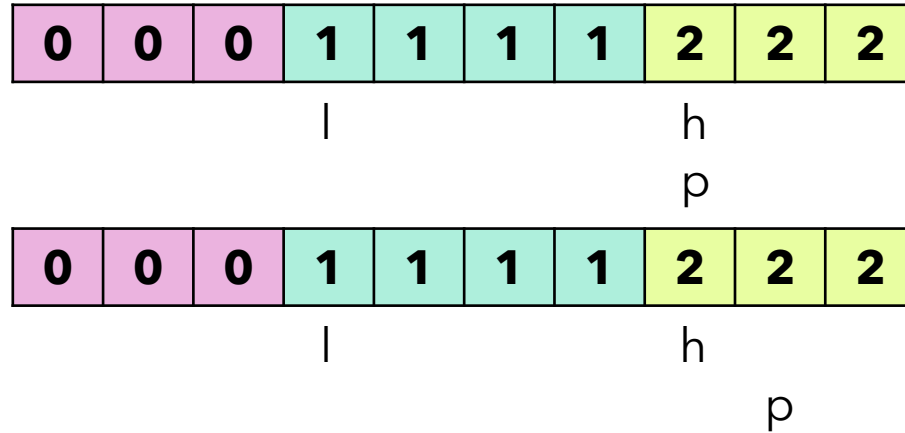


l

h

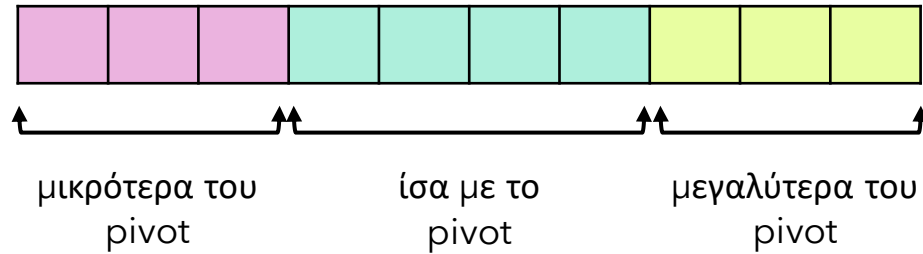
p

Άσκηση 6



Ο αλγόριθμος έχει πολυπλοκότητα $O(n)$ καθώς διατρέχει τα στοιχεία του πίνακα 1 φορά με οδηγό το ρινότ και χρήση των βοηθητικών δεικτών low, high για 1 ανταλλαγή και 2 πράξεις αύξησης / μείωσης των δεικτών το πολύ ανά βήμα. Δε χρησιμοποιείται επιπλέον μνήμη αλλά η ταξινόμηση γίνεται επιτόπου στον ήδη διαθέσιμο πίνακα.

- Ο αλγόριθμος γρήγορης ταξινόμησης, quicksort, χωρίζει τον πίνακα σε 2 μέρη.
- Θα μπορούσε να χωριστεί σε 3 μέρη, θεωρώντας πως στο πρώτο τμήμα αριστερά έχουν στοιχεία μικρότερα από το ρίβοτ, στο κέντρο στοιχεία ίσα με το ρίβοτ και δεξιά στοιχεία μεγαλύτερα από το ρίβοτ.



- Στη **χειρότερη** περίπτωση: όπως στον κλασικό quicksort, $\Theta(n^2)$
- Στη **μέση** περίπτωση: όπως στον κλασικό quicksort, $\Theta(n \log(n))$

4. Δίνεται πίνακας $A[1, \dots, n]$. Τα στοιχεία του αντιστοιχούν σε όρους αριθμητικής προόδου, διατεταγμένα κατά αύξουσα σειρά. Ένας όρος απουσιάζει. Δώστε έναν αποδοτικό αλγόριθμο για την εύρεση του 'χαμένου' όρου.

- Αρχικά εξετάζουμε το μεσαίο στοιχείο του πίνακα, $A[mid]$. Ελέγχουμε αν η διαφορά του από το προηγούμενο και το επόμενο στοιχείο είναι ίση με την κοινή διαφορά d .
- Αν όχι ο χαμένος όρος είναι ανάμεσα στο $A[mid]$ και $A[mid - 1]$ ή $A[mid + 1]$.
- Εάν ναι, τότε, αν το μεσαίο στοιχείο είναι ίσο με τον $(n/2)^{\text{στο}}$ όρο της αριθμητικής προόδου, ο χαμένος όρος βρίσκεται στον υποπίνακα $A[mid + 1, \dots, n]$ και συνεχίζουμε αναδρομικά σε αυτόν. Αλλιώς η αναδρομή συνεχίζει στον $A[1, \dots, mid - 1]$.

Ο αλγόριθμος μας είναι $O(\log n)$.

1	2	3	4	5	7	8	9	10
---	---	---	---	---	---	---	---	----

$$a_i = a_0 + (i-1)d$$

- $A[mid] = 5$
- $A[mid-1] = 4$
- $A[mid+1] = 7$
- $|A[mid] - A[mid-1]| = 1$
- **$|A[mid] - A[mid+1]| = 2$**

Έστω δύο πίνακες $X[1, \dots, n]$ και $Y[1, \dots, n]$, με καθένα να έχει n ταξινομημένους αριθμούς. Δώστε αλγόριθμο διαίρει και βασίλευε με χρόνο $O(\log n)$ για την εύρεση της διάμεσης τιμής των 2 πινάκων.

- Συγκρίνουμε τις διάμεσες τιμές των 2 πινάκων και για αυτόν με τη μικρότερη διάμεση τιμή, απορρίπτουμε το κομμάτι του πίνακα από την πρώτη θέση έως και 1 θέση πριν τη διάμεση, ενώ για αυτόν με τη μεγαλύτερη διάμεση τιμή, απορρίπτουμε το κομμάτι του πίνακα από την αμέσως επόμενη της διάμεσης θέση έως την τελευταία.
- Έτσι, οδηγούμαστε από είσοδο $(n + n) = 2n$ σε $\sim(n/2 + n/2) = n$
- Συνεχίζουμε έως να φτάσουμε σε 2 πίνακες με 2 θέσεις το πολύ.
- $T(n) = T(n/2) + O(1)$
 $a = 1$
 $b = 2$
 $f(n) = \Theta(n^{\log 1}) = \Theta(n^0) = \Theta(1) \Rightarrow \Theta.K. - \Pi 2: T(n) = O(\log n)$

X
1 2 3 4 5 27 28 29 30

Y
-1 -2 -3 -4 -5 17 18 19 20

*Με άρτιο n ,
 διάμεση τιμή (Δ)
 $= A[n/2+1]$

-1	-2	-3	-4	-5	1	2	3	4	5	17	18	19	20	27	28	29	30
----	----	----	----	----	---	---	---	---	---	----	----	----	----	----	----	----	----

X
1 2 3 4 5 27 28 29 30

Y
-1 -2 -3 -4 -5 17 18 19 20

$$\Delta x = 5 > \Delta y = -5$$

1	2	3	4	5
---	---	---	---	---

-5	17	18	19	20
----	----	----	----	----

$$\Delta x = 3 < \Delta y = 18$$

3	4	5
---	---	---

-5	17	18
----	----	----

$$\Delta x = 4 < \Delta y = 17$$

4	5
---	---

-5	17
----	----

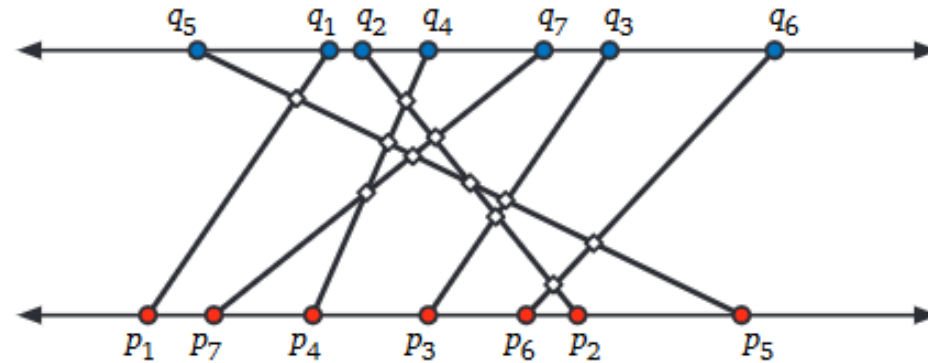
$$\Delta x = 5 < \Delta y = 17$$

4.5

$$-5 < 4 < 5 < 17$$

$$\Delta = (4+5)/2 = 4.5$$

Έχουμε 2 σύνολα n σημείων, ένα σύνολο $\{p_1, p_2, \dots, p_n\}$ στη γραμμή $y=0$ και ένα άλλο σύνολο $\{q_1, q_2, \dots, q_n\}$ στη γραμμή $y = 1$. Δημιουργούνται n ευθύγραμμα τμήματα μεταξύ των p_i και q_i . Περιγράψτε έναν αλγόριθμο διαίρει και βασίλευε που υπολογίζει πόσα ζεύγη αυτών των ευθύγραμμων τμημάτων τέμνονται, σε χρόνο $O(n \log n)$. Θεωρείται ότι τα p_i και q_i είναι διακριτές.



- Διατάσσουμε τα στοιχεία του πίνακα P με αύξοντα τρόπο και μεταθέτουμε αντίστοιχα τα στοιχεία του Q, ώστε το στοιχείο στην αρχική θέση Q_i να βρίσκεται στην αντίστοιχη νέα θέση του στοιχείου στη θέση P_i .
- Πλέον, για δείκτες θέσες $i < j$, τα ευθύγραμμα τμήματα τέμνονται αν και μόνο αν $Q[i] > Q[j]$. Συνεπώς, στόχος είναι να υπολογίσουμε τον αριθμό των αντιστροφών.
- Υπολογίζουμε τον αριθμό των αντιστροφών, επεκτείνοντας την ταξινόμηση με συγχώνευση:
 - χρωματίζουμε τα στοιχεία στο πρώτο μισό του Q, μπλε
 - χρωματίζουμε τα στοιχεία στο τελευταίο μισό του Q, κόκκινα
 - μετράμε αναδρομικά τις αντιστροφές και ταξινομούμε τα μπλε στοιχεία ($Q[1, \dots, \text{floor}(n/2)]$)
 - μετράμε αναδρομικά τις αντιστροφές και ταξινομούμε τα κόκκινα στοιχεία ($Q[\text{floor}(n/2)+1, \dots, n]$)
 - μετράμε τις αντιστροφές:
 - συγχωνεύοντας τους ταξινομημένους υποπίνακες $Q[1, \dots, \text{floor}(n/2)]$ και $Q[\text{floor}(n/2)+1, \dots, n]$, διατηρώντας τα αρχικά τους χρώματα
 - για κάθε μπλε στοιχείο $Q[i]$ τώρα, μετράμε τον αριθμό των μικρότερων στοιχείων, $Q[j]$, με κόκκινο χρώμα (CountRedBlue, $O(n)$)
- $T(n) = 2T(n/2) + O(n)$
 $a = 2$
 $b = 2$
 $f(n) = \Theta(n^{\log 2}) = \Theta(n^1) = \Theta(n) \Rightarrow \Theta.K. - \Pi 2: T(n) = O(n \log n)$

```

COUNTREDBLUE( $A[1..n]$ ):
  count  $\leftarrow 0$ 
  total  $\leftarrow 0$ 
  for i  $\leftarrow 1$  to n
    if  $A[i]$  is red
      count  $\leftarrow$  count + 1
    else
      total  $\leftarrow$  total + count
  return total

```

Λύστε τις ακόλουθες αναδρομικές εξισώσεις, προσδιορίζοντας την τάξη τους (Θ).
Σε όλες τις περιπτώσεις, υποθέτουμε πως $T(1) = 1$. Αιτιολογήστε τις απαντήσεις σας.

1.

2.

3.

4. $T(n) = 27T(\frac{n}{9}) + (\sqrt{n})^3 \lg(n)$

4. Θεώρημα κυριαρχίας - καλύπτει το Θ μόνο:

$$a = 27 \geq 1, b = 9 \geq 2, \log_b a = \log_9 27 = \frac{3}{2}, n^{\log_b a} = n^{\frac{3}{2}}$$

Η $f(n) = n^{\frac{3}{2}} \cdot \log n > 0$ για $n \rightarrow \infty$, δεν είναι πολυωνυμική, αλλά

$$f(n) = \Theta(n^{\frac{3}{2}} \cdot \log^1 n) = \Theta(n^{\log_b a} \cdot \log^k n), k = 1 \geq 0.$$

Έτσι, μπορεί να εφαρμοστεί το Θεώρημα Κυριαρχίας και πιο συγκεκριμένα η 'τέταρτη' περίπτωση, και κατά συνέπεια, $T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n) = \Theta(n^{1.5} \cdot \log^2 n)$.

Ακριβής λύση και Θ :

Αναδρομικό δέντρο:

$$T(n) = \sum_{i=0}^{\log_9(n)-1} (27^i (\frac{n}{9^i})^{1.5} \lg(\frac{n}{9^i})) + T(1) \cdot 27^{\log_9(n)} =$$

$$\sum_{i=0}^{\log_9(n)-1} (27^i (\frac{n^{1.5}}{27^i}) \lg(\frac{n}{9^i})) + 1 \cdot n^{\log_9(27)} =$$

$$n^{1.5} \sum_{i=0}^{\log_9(n)-1} (\lg(\frac{n}{9^i})) + n^{1.5} =$$

$$n^{1.5} \sum_{i=0}^{\log_9(n)-1} (\lg(n) - \lg(9^i)) + n^{1.5} =$$

$$n^{1.5} \sum_{i=0}^{\log_9(n)-1} (\lg(n) - i \lg(9)) + n^{1.5} =$$

$$n^{1.5} \lg(n) \sum_{i=0}^{\log_9(n)-1} (1) - n^{1.5} \lg(9) \sum_{i=0}^{\log_9(n)-1} (i) + n^{1.5} =$$

$$n^{1.5} (\lg(n) \frac{\lg(n)}{\lg(9)} - \lg(9) \frac{\lg(\frac{n}{9}) \lg(n)}{2 \lg(9) \lg(9)} + 1) =$$

$$n^{1.5} (\lg(n) \frac{\lg(n)}{\lg(9)} - \frac{\lg(\frac{n}{9}) \lg(n)}{2 \lg(9)} + 1) =$$

$$n^{1.5} (\lg(n) \frac{\lg(n)}{\lg(9)} - \frac{(\lg(n) - \lg(9)) \lg(n)}{2 \lg(9)} + 1) =$$

$$n^{1.5} (\frac{\lg(n)}{2 \lg(9)} (2 \lg(n) - \lg(n) + \lg(9)) + 1) =$$

$$n^{1.5} (\frac{\lg(n)}{2 \lg(9)} (\lg(n) + \lg(9)) + 1) =$$

$$n^{1.5} (\frac{\lg(n)}{2 \lg(9)} (\lg(9n)) + 1) =$$

$$n^{1.5} (\frac{\lg(n) \lg(9n)}{2 \lg(9)} + 1) =$$

$$n^{1.5} (\frac{\lg(n) \lg(9n)}{\lg(9^2)} + 1) \Rightarrow$$

$$T(n) = n^{1.5} (\frac{\lg(n) \lg(9n)}{\lg(81)} + 1) = \Theta(n^{1.5} \cdot \log^2 n)$$

ή

Μέθοδος αντικατάστασης:

Το n είναι δύναμη του 9 ($n = 9^i, i \in \mathbb{Z}^+$)

$$T(1) = T(9^0) = 1$$

$$\text{Υπόθεση: } T(n) = n^{1.5} (\frac{\lg(n) \lg(9n)}{\lg(81)} + 1)$$

Επαγωγική βάση: Για $n = 9^0 = 1, T(1) = 1^{1.5} (\frac{\lg(1) \lg(9)}{\lg(81)} + 1) = 1(0 + 1) = 1$, που ισχύει

Επαγωγική υπόθεση: Για $\forall k \leq 1, T(k) = k^{1.5} (\frac{\lg(k) \lg(9k)}{\lg(81)} + 1)$

Επαγωγικό βήμα: $T(9k) = 27T(k) + (9k)^{1.5} \lg(9k) =$

$$27(k^{1.5} (\frac{\lg(k) \lg(9k)}{\lg(81)} + 1)) + (9k)^{1.5} \lg(9k) =$$

$$(9k)^{1.5} \frac{\lg(k) \lg(9k)}{\lg(81)} + (9k)^{1.5} + (9k)^{1.5} \lg(9k) =$$

$$(9k)^{1.5} (\frac{\lg(k) \lg(9k)}{\lg(81)} + \lg(9k) + 1) =$$

$$(9k)^{1.5} (\lg(9k) (\frac{\lg(k)}{\lg(81)} + 1) + 1) =$$

$$(9k)^{1.5} (\lg(9k) \frac{\lg(k) + \lg(81)}{\lg(81)} + 1) =$$

$$(9k)^{1.5} (\lg(9k) \frac{\lg(81k)}{\lg(81)} + 1) =$$

$$(9k)^{1.5} (\lg(9k) \frac{\lg(9(9k))}{\lg(81)} + 1) =$$

$$(9k)^{1.5} (\frac{\lg(9k) \lg(9(9k))}{\lg(81)} + 1), \text{ άρα αποδείχθηκε το ζητούμενο}$$

$$\text{Έτσι, } T(n) = (9k)^{1.5} (\frac{\lg(9k) \lg(9(9k))}{\lg(81)} + 1) = \Theta(n^{1.5} \cdot \log^2 n).$$

ΣΥΝΕΧΕΙΑ ΣΤΟ ΕΠΟΜΕΝΟ ΦΡΟΝΤΙΣΤΗΡΙΟ...



Πηγές Φ4:

Σημειώσεις / διαλέξεις μαθήματος, προτεινόμενα βιβλία, υλικό μαθήματος προηγούμενων ετών, Michael Sambol, SMA5503, CSE235